2026-8-4 Linux进程信号--概念

- 1. 信号的基本概念与生命周期
  - 1.1 信号的基本认识
    - 1.1.1 从生活中的“通知”理解信号
    - 1.1.2 信号在系统中的含义
    - 1.1.3 信号与信号量不是同一个概念
  - 1.2 信号的完整生命周期
    - 1.2.1 产生、保存、处理
    - 1.2.2 为什么信号通常不会立即处理
    - 1.2.3 信号保存在哪里
- 2. 信号编号、名称和默认动作
  - 2.1 从编号到处理动作
  - 2.2 默认动作不等于“一定终止”
- 3. 信号的产生
  - 3.1 从终端键盘产生
  - 3.2 使用 kill 命令或 kill() 函数产生
  - 3.3 进程给自己发送信号：raise()
  - 3.4 使用 abort() 产生 SIGABRT
  - 3.5 由软件条件产生
  - 3.6 由硬件异常产生
- 4. 进程终止与信号处理
  - 4.1 正常退出和被信号终止
  - 4.2 core dump 是什么
  - 4.3 用 ulimit 控制 core 文件
  - 4.4 默认、忽略和捕获
  - 4.5 signal() 接口只设置处理方式
  - 4.6 信号处理发生时的执行流切换
  - 4.7 最小处理示例
- 5. alarm 定时器与 pause
  - 5.1 alarm() 的基本语义
  - 5.2 一次定时与重复定时
  - 5.3 操作系统如何管理定时器
- 6. 全链路总结
  - 6.1 从事件到结果的闭环
  - 6.2 理解时最容易混淆的几组关系

1. 信号的基本概念与生命周期

1.1 信号的基本认识

1.1.1 从生活中的“通知”理解信号

红绿灯、闹钟、门铃、手机铃声都表达某种已经约定好的事件。人看到或听到它们以后，会按照已有经验采取动作。进程信号也是类似的机制，只是参与者变成了进程、操作系统和外部事件。
可以抓住三个要点：

1. 信号本身用来通知“某件事发生了”，不是直接替进程完成处理。
2. 处理方式不是收到信号时才临时决定，而是提前准备好的：使用默认动作、忽略信号，或者注册自定义处理函数。
3. 进程能够识别和处理信号，是操作系统内核提供的内置能力；普通程序面对的是信号编号、默认动作和信号接口。
   例如，人在工作时不一定会立刻接电话；进程收到信号时也不一定立刻执行处理动作。它可能正在执行更紧急的内核操作或用户代码，内核会选择一个合适的时机再处理。

1.1.2 信号在系统中的含义

Linux 信号是外部事件、其他进程、软件条件或硬件异常向进程发送的一种异步事件通知机制，也可以理解为面向进程的“软中断”。
它的几个关键特点是：

- **面向进程**：硬件中断通常先发给 CPU，再由操作系统处理；信号则最终投递给目标进程。
- **异步**：信号可能在进程执行任意一段用户代码时到来，程序不能假设它只会出现在某个固定位置。
- **携带的信息有限**：普通信号最核心的信息是信号编号。编号说明事件类型，具体怎么处理由目标进程当前登记的动作决定。
- **最终由内核投递**：不管信号来源是键盘、命令、函数还是硬件异常，真正把信号记录到目标进程中的都是操作系统。
  所以“给进程发信号”的本质，是请求内核修改目标进程的信号待处理信息，而不是调用者直接修改另一个进程的用户空间数据。

1.1.3 信号与信号量不是同一个概念

“信号（signal）”和“信号量（semaphore）”只是中文名称相近，二者没有直接关系。

- 信号用于通知异步事件，属于进程事件机制。
- 信号量用于同步和互斥，表示资源数量或临界区状态。
  学习本章时不能把两者混用。

1.2 信号的完整生命周期

1.2.1 产生、保存、处理

一个信号从事件发生到处理完成，至少经过三个阶段：

阶段	发生什么	关键理解
信号产生	键盘、进程、软件条件或硬件异常触发信号	各种来源最终都要经过内核
信号保存	内核在目标进程中记录待处理信号	信号已经产生，但尚未递送给处理动作
信号处理	进程执行默认动作、忽略信号或运行自定义处理函数	处理动作由信号编号和目标进程的登记决定

这条时间线解释了为什么“发送信号”和“处理信号”不是同一时刻发生的两个动作。

1.2.2 为什么信号通常不会立即处理

信号到达时，目标进程可能正处在不适宜切换处理路径的位置，例如正在执行关键内核逻辑，或者当前工作不能被随意打断。内核需要先在进程控制信息中记录信号，等进入一个安全、合适的返回点后，再决定是否递送。
因此可以形成下面的关系：

```
事件发生
    |
    v
内核记录待处理信号
    |
    v
进程到达合适的处理时机
    |
    v
执行默认动作 / 忽略 / 自定义处理函数
```

“异步”意味着信号可以在程序运行途中到达；“延迟处理”意味着信号不必在事件产生的瞬间插入到进程的正常执行流中。

1.2.3 信号保存在哪里

长图用位图描述信号的保存方式：每个信号编号对应一个二进制位，位的位置表示信号编号，位的值表示该信号是否处于待处理状态。
对整体机制可以这样理解：

- 进程的 `task_struct` 中包含信号相关的数据结构。
- 内核通过待处理集合记录“已经产生、还要交付给进程”的信号。
- 修改该集合本质上是在修改内核中的进程控制信息，普通用户代码不能直接改写。
- 多个来源发送信号时，最终都由内核完成记录和递送。
  信号编号 2 对应 `SIGINT`，编号 11 对应 `SIGSEGV`。看到待处理位图中有对应位，表示该信号已经到达内核记录阶段，但还没有完成最终处理。

2. 信号编号、名称和默认动作

2.1 从编号到处理动作

每个标准信号都有编号、宏名称和默认动作。常见默认动作可以归为以下几类：

- `Term`：终止进程。
- `Core`：终止进程，并允许生成 core 文件。
- `Stop`：暂停进程。
- `Cont`：继续运行被暂停的进程。
- `Ign`：忽略信号。
  可以通过 `kill -l` 查看系统中的信号列表，通过 `man 7 signal` 查看各信号的含义和默认动作。程序源码通常使用 `SIGINT`、`SIGQUIT` 等宏名称；内核和命令行最终仍然把它转换为信号编号处理。
  常见标准信号如下（以常见的 x86 Linux 环境为例，具体编号最终以本机 `kill -l` 和 `man 7 signal` 为准）：

编号	名称	默认动作	主要含义
1	`SIGHUP`	Term	控制终端挂断或控制进程结束
2	`SIGINT`	Term	键盘中断，通常由 `Ctrl+C` 产生
3	`SIGQUIT`	Core	键盘退出，通常由 `Ctrl+\` 产生

编号	名称	默认动作	主要含义
4	`SIGILL`	Core	非法指令
5	`SIGTRAP`	Core	调试断点或陷阱
6	`SIGABRT`	Core	`abort()` 产生的异常终止
7	`SIGBUS`	Core	总线错误，例如错误的内存访问
8	`SIGFPE`	Core	算术异常，例如除零
9	`SIGKILL`	Term	强制终止，不可捕获、不可忽略
10	`SIGUSR1`	Term	用户自定义信号 1
11	`SIGSEGV`	Core	非法内存访问，例如空指针解引用
12	`SIGUSR2`	Term	用户自定义信号 2
13	`SIGPIPE`	Term	向没有读端的管道或套接字写入
14	`SIGALRM`	Term	`alarm()` 设置的定时器超时
15	`SIGTERM`	Term	请求进程正常终止
16	`SIGSTKFLT`	Term	协处理器栈错误
17	`SIGCHLD`	Ign	子进程停止或终止
18	`SIGCONT`	Cont	继续运行被暂停的进程
19	`SIGSTOP`	Stop	强制暂停，不可捕获、不可忽略
20	`SIGTSTP`	Stop	终端暂停，通常由 `Ctrl+Z` 产生
21	`SIGTTIN`	Stop	后台进程读取终端
22	`SIGTTOU`	Stop	后台进程写入终端
23	`SIGURG`	Ign	套接字上有紧急数据
24	`SIGXCPU`	Core	CPU 时间限制超限
25	`SIGXFSZ`	Core	文件大小限制超限
26	`SIGVTALRM`	Term	虚拟定时器超时
27	`SIGPROF`	Term	性能分析定时器超时
28	`SIGWINCH`	Ign	终端窗口大小改变
29	`SIGIO`	Term	可执行 I/O 事件
30	`SIGPWR`	Term	电源故障
31	`SIGSYS`	Core	非法系统调用

Linux 还提供 34 以上的实时信号；本笔记的长图主线只展开常见标准信号及其产生、保存和处理机制。

2.2 默认动作不等于“一定终止”

“大多数信号的默认动作是终止进程”是一种概括，但不能套用到所有信号：

- `SIGCHLD`、`SIGURG`、`SIGWINCH` 默认可能被忽略。
- `SIGSTOP`、`SIGTSTP`、`SIGTTIN`、`SIGTTOU` 默认是暂停。
- `SIGCONT` 默认是继续运行。
- `SIGSEGV`、`SIGFPE` 等默认动作还会带有 core dump 标记。
  因此，判断一个信号的最终效果时，既要看信号编号，也要看该信号的默认动作以及进程是否修改过处理方式。

3. 信号的产生

3.1 从终端键盘产生

在终端中按下组合键时，键盘先产生硬件输入事件，内核接收并解释它，然后把对应信号发送给当前前台进程组。

按键	信号	编号	典型结果
`Ctrl+C`	`SIGINT`	2	终止前台进程
`Ctrl+\`	`SIGQUIT`	3	终止前台进程，并允许产生 core 文件
`Ctrl+Z`	`SIGTSTP`	20	暂停前台进程，交给作业控制

这类终端控制信号只面向当前前台进程组。使用 `&` 放到后台运行的进程不会因为用户在前台按 `Ctrl+C` 而被终止；需要时可以用 `kill` 按 PID 给它发送信号。
被 `Ctrl+Z` 暂停的进程并没有退出。它等待 `SIGCONT`（18）继续运行，之后可以由 Shell 的作业控制把它重新放到前台或后台。
键盘事件、硬件中断和进程信号处在不同层次：

```
按键
  -> 键盘硬件产生输入事件
  -> CPU/内核接收中断
  -> 终端和作业控制识别组合键
  -> 内核向目标进程记录信号
```

3.2 使用 kill 命令或 kill() 函数产生

Shell 中的 `kill` 命令最终还是通过 `kill()` 接口请求内核发送信号：

```c
#include <sys/types.h>
#include <signal.h>

int kill(pid_t pid, int sig);
```

- `pid` 指定目标进程。
- `sig` 指定信号编号或对应宏。
- 成功发送至少一个信号时通常返回 0；失败时返回 -1，并设置 `errno`。
  命令和函数可以表达同一个动作：

```bash
kill -SIGTERM 12345
kill -15 12345

kill -SIGSEGV 12345
kill -11 12345
```

这里要注意：`kill -SIGSEGV` 是“向进程发送 11 号信号”，进程因为 `SIGSEGV` 的默认动作而终止；它不一定真的执行了非法内存访问。Shell 显示 `Segmentation fault` 描述的是进程收到信号后的表现，而不是发送命令一定造成了内存错误。

3.3 进程给自己发送信号：raise()

`raise()` 用于向调用它的进程或线程发送信号，概念上可以理解为：

```c
raise(sig);
kill(getpid(), sig);
```

它与 `kill()` 的区别主要在于目标：

- `kill()` 可以向另一个指定进程发送信号。
- `raise()` 直接向调用者自身发送信号。
  `raise()` 成功时返回 0，失败时返回非零值。

3.4 使用 abort() 产生 SIGABRT

`abort()` 用于使当前进程异常终止，通常会发送 6 号信号 `SIGABRT`：

```c
#include <stdlib.h>

void abort(void);
```

`abort()` 不会正常返回。即使进程注册了 `SIGABRT` 的自定义处理函数，处理函数执行结束以后，`abort()` 仍会继续异常终止流程，而不是让程序像正常 `return` 一样继续运行。

3.5 由软件条件产生

软件条件信号来自操作系统检测到的软件状态或系统事件。长图重点举例了两类：

- `SIGPIPE`（13）：向一个已经没有读端的管道或套接字继续写入时产生，默认终止进程。
- `SIGALRM`（14）：通过 `alarm()` 设置的定时器到期后产生，默认终止进程。
  这里同样遵循统一模型：软件代码或内核检测到条件，内核记录相应信号，目标进程再按照已登记的处理方式响应。

3.6 由硬件异常产生

CPU、MMU 等硬件执行指令时发现异常，会通过异常或中断路径把控制权交给内核。内核判断异常与当前进程有关，再把这种硬件异常解释成进程信号。

场景	硬件层现象	对应信号
整数除零	CPU 算术异常	`SIGFPE`（8）
访问非法地址、空指针	MMU/页表检查失败	`SIGSEGV`（11）
执行非法指令	CPU 指令异常	`SIGILL`（4）
地址总线相关问题	总线异常	`SIGBUS`（7）

这类信号与“别人主动通知我”不同，它和当前执行的指令直接相关，属于同步异常。若处理函数只是打印信息后直接返回，原故障条件仍然存在，CPU 可能重新执行同一条故障指令，于是再次产生信号，表现为反复收到同一个信号。
所以捕获 `SIGFPE` 或 `SIGSEGV` 并不等于修复了除零或非法地址。处理函数若要继续运行，必须消除故障条件或修正执行上下文；否则更合理的做法是让进程终止并保留现场进行调试。

4. 进程终止与信号处理

4.1 正常退出和被信号终止

父进程通过 `wait()` 或 `waitpid()` 回收子进程时，可以从状态字判断子进程为何结束。

```
pid_t waitpid(pid_t pid, int *wstatus, int options);
```

例如：

```
int status = 0;
waitpid(-1, &status, 0);
```

状态字的概念结构可以概括为：

结束方式	状态字的含义
正常退出	低 7 位为 0，高字节保存退出状态
被信号终止	低 7 位保存终止信号编号，另有 core dump 标志

实际编程应使用系统提供的宏，而不是手写位移：

```
#include <sys/wait.h>

if (WIFEXITED(status)) {
    int code = WEXITSTATUS(status);
} else if (WIFSIGNALED(status)) {
    int sig = WTERMSIG(status);
    int dumped = WCOREDUMP(status);
}
```

这样就能把“子进程正常调用 `exit()` 或从 `main()` 返回”与“子进程被信号终止”区分开。

4.2 core dump 是什么

当一个进程因为 `Core` 类信号异常终止时，系统可以把该进程的用户空间内存等现场信息保存到磁盘，形成 core 文件。它主要用于事后调试，确认程序在异常发生时的调用栈、寄存器状态和内存内容。
需要区分两个层面：

- `SIGINT` 默认是 `Term`，通常只终止进程。
- `SIGQUIT`、`SIGFPE`、`SIGSEGV`、`SIGABRT` 等默认是 `Core`，允许在终止进程的同时生成 core 文件。
  “允许生成”不表示一定会生成成功。还需要满足 core 文件大小限制、文件系统权限和系统的 core 文件命名策略。

4.3 用 ulimit 控制 core 文件

Shell 的资源限制会被它启动的子进程继承。查看限制：

```
ulimit -a
```

关闭 core 文件：

```
ulimit -c 0
```

允许生成，例如把大小限制设为 1024 个块：

```
ulimit -c 1024
```

之后从同一个 Shell 启动的测试进程就会继承该限制。有些服务器环境默认把 core 文件大小设为 0，目的是避免磁盘占用和信息泄露。

4.4 默认、忽略和捕获

进程可以采用三种基本方式处理信号：

方式	设置	行为
默认动作	`SIG_DFL`	按系统为该信号定义的默认动作处理
忽略信号	`SIG_IGN`	收到后不执行默认终止等动作
自定义捕获	注册处理函数	让进程执行自己的 `handler(signum)`

自定义捕获通常也叫“捕获（catch）”信号。`SIG_DFL` 和 `SIG_IGN` 可以理解为两个特殊的处理动作，系统会把它们和普通函数指针一起记录。

并非所有信号都能修改：

- `SIGKILL`（9）不能被捕获或忽略。
- `SIGSTOP`（19）不能被捕获或忽略。

因此程序不能用自定义处理函数强行阻止 `SIGKILL` 或 `SIGSTOP`。

4.5 signal() 接口只设置处理方式

基本的信号登记接口是：

```
#include <signal.h>

typedef void (*sighandler_t)(int);

sighandler_t signal(int signum, sighandler_t handler);
```

- `signum` 是信号编号。
- `handler` 是处理函数、`SIG_DFL` 或 `SIG_IGN`。
- 自定义处理函数接收一个 `int`，其中是收到的信号编号。

调用：

```
signal(SIGINT, handler);
```

只是告诉内核“以后收到 `SIGINT` 时调用 `handler`”。这句话本身不会调用 `handler`；如果之后没有对应信号到来，处理函数永远不会执行。

把处理方式设为默认或忽略：

```
signal(SIGINT, SIG_DFL);
signal(SIGINT, SIG_IGN);
```

4.6 信号处理发生时的执行流切换

以 `Ctrl+C` 为例，完整路径可以写成：

```
进程正常执行
  -> 键盘产生输入事件
  -> 内核识别 Ctrl+C
  -> 向当前前台进程记录 SIGINT(2)
  -> 进程到达安全的返回点
  -> 执行默认动作、忽略动作或 handler
  -> 继续原执行流或终止进程
```

如果执行的是自定义处理函数，内核需要在用户态处理函数和原程序执行流之间保存、恢复上下文。处理函数返回后，通常继续执行原先被信号打断的位置；如果该信号的动作是终止进程，则不会返回原流程。

这也解释了“信号相对于进程控制流是异步的”：处理函数可能插入到普通代码执行过程中的任何可交付时刻。

4.7 最小处理示例

下面的例子先登记 `SIGINT` 处理器，再让进程等待信号：

```c
#include <signal.h>
#include <unistd.h>

volatile sig_atomic_t received = 0;

void handler(int signo)
{
    received = signo;
}

int main(void)
{
    signal(SIGINT, handler);

    while (!received)
        pause();

    return received == SIGINT ? 0 : 1;
}
```

运行后按 `Ctrl+C`，进程不再执行 `SIGINT` 的默认终止动作，而是进入 `handler()`，记录编号 2，随后从 `pause()` 返回并结束循环。

5. alarm 定时器与 pause

5.1 alarm() 的基本语义

```c
#include <unistd.h>

unsigned int alarm(unsigned int seconds);
```

`alarm(seconds)` 请求内核在经过 `seconds` 秒后向调用进程发送 `SIGALRM`：

- `seconds > 0`：设置一个一次性定时器。
- `alarm(0)`：取消之前设置的定时器。
- 若之前已有未到期的定时器，新调用会替换旧定时器。

返回值是“上一个定时器还剩多少秒”；如果此前没有定时器，则返回 0。返回值不是新定时器的完整时间。

`SIGALRM` 的默认动作是终止进程。若要利用闹钟执行自定义操作，必须先登记处理函数，再调用 `alarm()`。

5.2 一次定时与重复定时

```c
#include <signal.h>
#include <unistd.h>

volatile sig_atomic_t ticks = 0;

void on_alarm(int signo)
{
    (void)signo;
    ++ticks;
    alarm(1);        // 在处理函数中重新设置下一次闹钟
}

int main(void)
{
    signal(SIGALRM, on_alarm);
    alarm(1);

    while (1)
```

```
        pause();
}
```

理解这个例子要同时抓住两条线：

1. `alarm(1)` 只生效一次；自定义处理函数返回后，不会自动产生下一次闹钟。
2. 在 `SIGALRM` 处理函数中再次调用 `alarm(1)`，才能形成周期触发。

`pause()` 会使调用进程睡眠，直到有信号导致处理函数运行或进程终止。捕获信号并执行完处理函数后，`pause()` 通常返回 -1，并把 `errno` 设为 `EINTR`，表示等待被信号中断。

如果没有捕获 `SIGALRM`，进程会直接采用默认动作终止，而不是执行上面的计时逻辑。

5.3 操作系统如何管理定时器

操作系统本身必须具有计时能力，才能实现闹钟、超时和调度等功能。某个进程调用 `alarm()` 时，只是向内核登记了一项定时任务；系统需要描述并组织这些任务。

长图用类似下面的结构说明定时器的核心信息：

```
struct timer {
    int expires;                    // 何时到期
    int cnt;                        // 需要触发多少次
    void (*callback)(void);         // 到期后执行什么动作
};
```

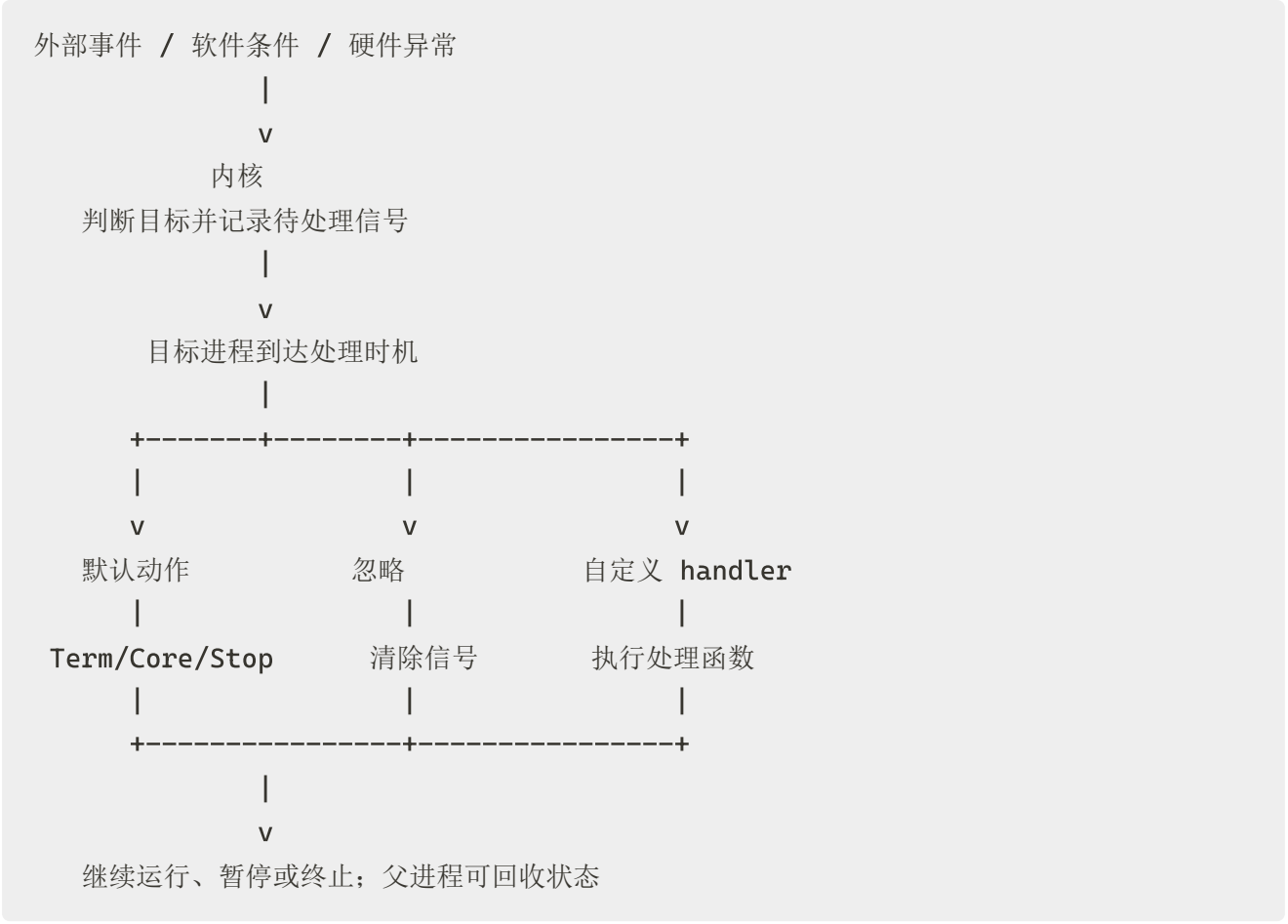
从概念上看，定时器至少需要回答三个问题：

- 什么时间到期：`expires`。
- 到期时做什么：`callback` 或对应的内核处理逻辑。
- 多个定时器怎样组织：操作系统会使用适合的管理结构，例如按到期时间组织成时间轮或堆。

因此，`alarm()` 的完整过程是：用户登记超时 -> 内核保存定时任务 -> 到期时产生 `SIGALRM` -> 目标进程按默认动作或自定义处理函数响应。

6. 全链路总结

6.1 从事件到结果的闭环



可以用一组对应关系把主线串起来：

触发来源	典型信号	直接结果
`Ctrl+C`	`SIGINT`（2）	默认终止前台进程
`Ctrl+\`	`SIGQUIT`（3）	默认终止并允许 core dump
`Ctrl+Z`	`SIGTSTP`（20）	默认暂停进程
`kill()` / `kill` 命令	指定编号	由目标信号的登记动作决定
`raise()`	指定编号	向调用者自身发送
`abort()`	`SIGABRT`（6）	异常终止
向无读端管道写	`SIGPIPE`（13）	默认终止
`alarm()` 到期	`SIGALRM`（14）	默认终止；可捕获后继续
除零	`SIGFPE`（8）	默认终止并允许 core dump
非法内存访问	`SIGSEGV`（11）	默认终止并允许 core dump

6.2 理解时最容易混淆的几组关系

- **信号与信号量**：名称相近，但一个用于事件通知，一个用于同步互斥。
- **产生与处理**：产生信号不等于立刻执行处理动作，中间还存在内核保存和选择处理时机的阶段。
- **登记与调用**：`signal()` 只登记未来的处理方式，不会主动触发处理函数。
- **硬件中断与进程信号**：二者都体现“打断当前流程并通知事件”，但硬件中断面向 CPU，信号面向进程。
- **终止与 core dump**：`Core` 同时包含终止和保存现场两层含义；core 文件生成还受资源限制。
- **捕获与修复**：捕获 `SIGFPE`、`SIGSEGV` 只是接管了响应方式，若原始故障仍在，程序可能再次收到同一信号。